# Web Accessibility (A11y)



1- Why accessibility matters

2- WCAG guidelines

3- Semantic HTML

4- ARIA basics

5- Keyboard navigation

6- Testing tools

**Let's build for everyone!**

SCROLL

# Web Accessibility (A11y)

**WHY "A11y"? A + 11 letters + y = Accessibility (Tech shorthand, like i18n for internationalization)**

**TODAY'S MISSION:** Making the web accessible to ALL users, regardless of ability

**WHAT WE'RE LEARNING:**

- Why accessibility is legally and ethically required
- WCAG standards and guidelines
- Practical implementation techniques
- Testing tools and methods




**15% of the world has a disability—that's over 1 BILLION people!**

# Today's Agenda

## Today's Accessibility Roadmap

**PART 1: WHY ACCESSIBILITY MATTERS**
- Statistics and demographics
- Legal requirements (ADA, WCAG)
- Ethical responsibility
- Business benefits

**PART 2: WCAG GUIDELINES**
- The POUR principles
- Success criteria levels (A, AA, AAA)
- Key requirements

**PART 3: SEMANTIC HTML**
- Why structure equals accessibility
- Proper element usage
- Heading hierarchy
- Links and buttons

**PART 4: ARIA BASICS**
- When to use ARIA (and when NOT to!)
- Roles, states, properties
- Live regions

**PART 5: KEYBOARD NAVIGATION**
- Why keyboard access matters
- Focus management
- Tab order and skip links

**PART 6: TESTING TOOLS**
- WAVE, axe DevTools
- Screen reader demo
- Manual testing

# Accessibility Affects EVERYONE

**THE DISABILITY STATISTICS:**

- **1 in 4 US adults** has a disability *That's 61 MILLION Americans*
- **15% of world population** has some form of disability *Over 1 BILLION people globally*
- **8% of men** are colorblind *12 million in the US alone—more than NYC population!*
- **26% of adults 65+** have a disability *Aging population = growing need*
- **100% of us** will experience temporary or situational limitations *Broken arm, bright sunlight, noisy cafe, holding a baby*

The web is FOR everyone. Let's make it WORK for everyone.




**THIS ISN'T A NICHE. THIS IS YOUR AUDIENCE.**

# Types of Disabilities

## Who Are We Designing For?

**VISUAL DISABILITIES**
- Blindness (total vision loss)
- Low vision (partial vision, blurry)
- Colorblindness (can't distinguish certain colors)
- Light sensitivity

**AUDITORY DISABILITIES**
- Deafness (total hearing loss)
- Hard of hearing (partial hearing loss)
- Auditory processing disorders

**MOTOR/MOBILITY DISABILITIES**
- Limited mobility (can't use mouse)
- Tremors or spasms
- Paralysis
- Missing limbs
- Arthritis

**COGNITIVE DISABILITIES**
- Learning disabilities (dyslexia, dyscalculia)
- Memory impairments
- Attention disorders (ADHD)
- Autism spectrum

**SEIZURE DISORDERS**
- Photosensitive epilepsy (triggered by flashing)

**+ TEMPORARY & SITUATIONAL LIMITATIONS!** *Broken arm, noisy room, bright sunlight, holding baby*

**Design for permanent disabilities = helps EVERYONE**

# Legal Requirements

## Accessibility is THE LAW

### UNITED STATES LAWS:

**ADA (Americans with Disabilities Act)** - 1990

- Title III: Public accommodations (includes websites!)
- Courts have ruled websites ARE public accommodations
- Applies to: Businesses, schools, government

**Section 508** - 1998, updated 2017

- Federal agencies MUST be accessible
- Government contractors MUST be accessible
- Based on WCAG 2.0 Level AA

### INTERNATIONAL LAWS:

**WCAG 2.1** (Web Content Accessibility Guidelines)

- International standard by W3C
- Level AA = typical legal requirement
- Adopted by US, EU, Canada, Australia, many others

**EU Accessibility Act** - Effective 2025

- Requires accessibility for products/services in EU
- Includes websites and mobile apps

**UK Equality Act** - 2010

- Websites must be "perceivable, operable, understandable"

**Canadian Standard (AODA)** - Ontario

- WCAG 2.0 Level AA required

**YOUR SITES CAN BE SUED FOR NON-COMPLIANCE**

# Real Lawsuits

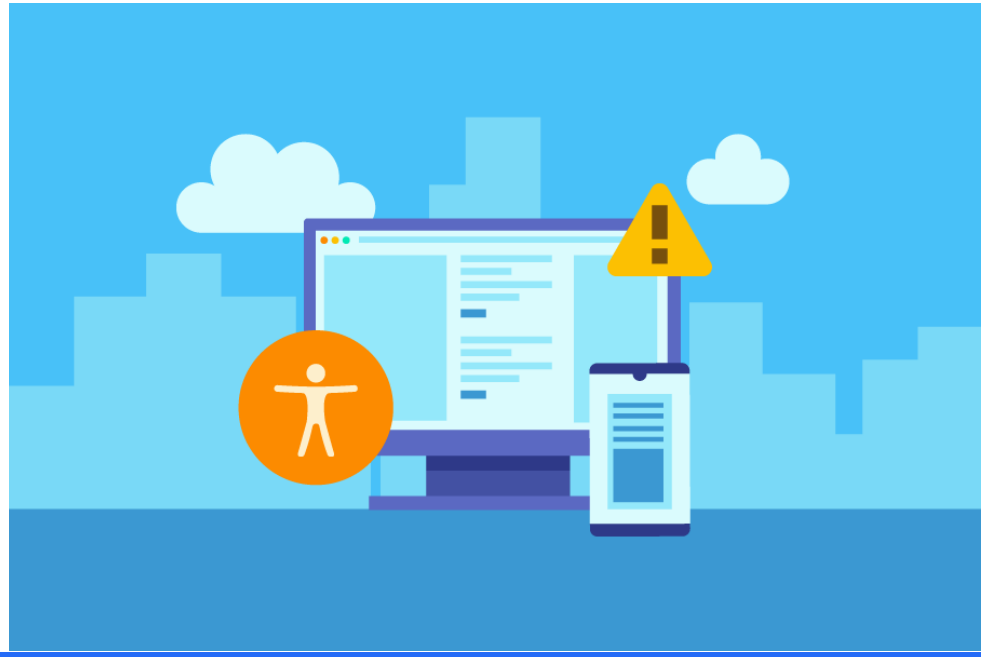
## It Happens to BIG Companies

**Target** (2008)
- $6 million settlement
- Couldn't add items to cart with screen reader
- 3 years of litigation

**Netflix** (2012)
- $755,000 settlement
- No captions on streaming content
- Also required to caption ALL content going forward

**Domino's Pizza** (2019)
- Went to Supreme Court
- Blind user couldn't order pizza on website/app
- Ruled ADA applies to websites
- **Game-changing precedent**

**Beyoncé/Parkwood Entertainment** (2019)
- Sued for website inaccessibility
- Settled, website now accessible



**Your university? Can be sued. Your client? Can be sued. Your employer? Can be sued.**

# Beyond Legal - It's Ethical

## It's Simply the RIGHT Thing to Do

***"The power of the Web is in its universality. Access by everyone regardless of disability is an essential aspect."***
**— Tim Berners-Lee** *Inventor of the World Wide Web*

### THE ETHICAL ARGUMENT:

**Equal Access = Human Right** The UN Convention on Rights of Persons with Disabilities recognizes web access as a human right

**Digital Exclusion = Real Exclusion** In 2026, if you can't access the web, you can't:

- Apply for jobs (most are online)
- Access government services
- Shop (especially in pandemic era)
- Learn (online education)
- Connect socially
- Access healthcare info
- Bank
- Basically participate in modern society

**The web was invented to be universal. Let's keep it that way.**

# Business Benefits

## Accessibility is GOOD Business

**EXPAND YOUR AUDIENCE**

- Don't exclude 15-26% of potential customers!
- 61 million Americans = massive market
- Global disability market: $8 TRILLION in annual disposable income

**IMPROVE SEO**

- Google rewards accessible sites
- Semantic HTML = better crawlability
- Alt text = image search optimization
- Captions = video searchability
- **Accessible sites rank higher**

**ENHANCE UX FOR EVERYONE**

- Keyboard shortcuts = faster for power users
- Captions = better in noisy/quiet environments
- Clear language = better for non-native speakers
- Good contrast = readable for everyone
- **Better UX = higher engagement = more conversions**

**FUTURE-PROOF YOUR SITE**

- Works with new devices/browsers
- Compatible with voice assistants
- Ready for AR/VR accessibility
- Clean semantic code = easier to maintain

**BUILD BRAND REPUTATION**

- Shows you care about ALL users
- Positive PR and customer loyalty
- Competitive advantage
- Attracts talent (developers want to work on ethical projects)

**Accessibility = Larger reach + Better performance + Legal protection**

# The Curb Cut Effect

**THE STORY:** Curb cuts were mandated for wheelchair accessibility in the 1970s.

**BUT THEY HELP:**

- Wheelchair users (intended audience)
- Parents with strollers
- Travelers with luggage
- Cyclists
- Skateboarders
- Delivery carts
- Everyone who finds steps inconvenient

**SAME WITH WEB ACCESSIBILITY:**

**Captions** → Help deaf users + noisy environments + quiet spaces + ESL learners

**Keyboard nav** → Help motor disabilities + power users (it's faster!)

**Good contrast** → Help low vision + bright sunlight + older users

**Clear language** → Help cognitive disabilities + ESL + everyone in a hurry

**Design for the most constrained case, and EVERYONE benefits!**

# What is WCAG?

## Web Content Accessibility Guidelines (WCAG)

**WHAT IS WCAG?** International standard by W3C (World Wide Web Consortium)

- Current version: **WCAG 2.1** (2018)
- Version 2.2 coming soon
- Version 3.0 in development

**THREE CONFORMANCE LEVELS:**

- **Level A** - Minimum basic accessibility *If you don't meet this, major barriers exist*
- **Level AA** - Recommended target ✓ *Legally compliant, achievable for most sites*
- **Level AAA** - Highest level *Very strict, not always possible for all content*

**YOUR TARGET: WCAG 2.1 Level AA: This is what laws require, what courts expect, and what's professionally achievable.**

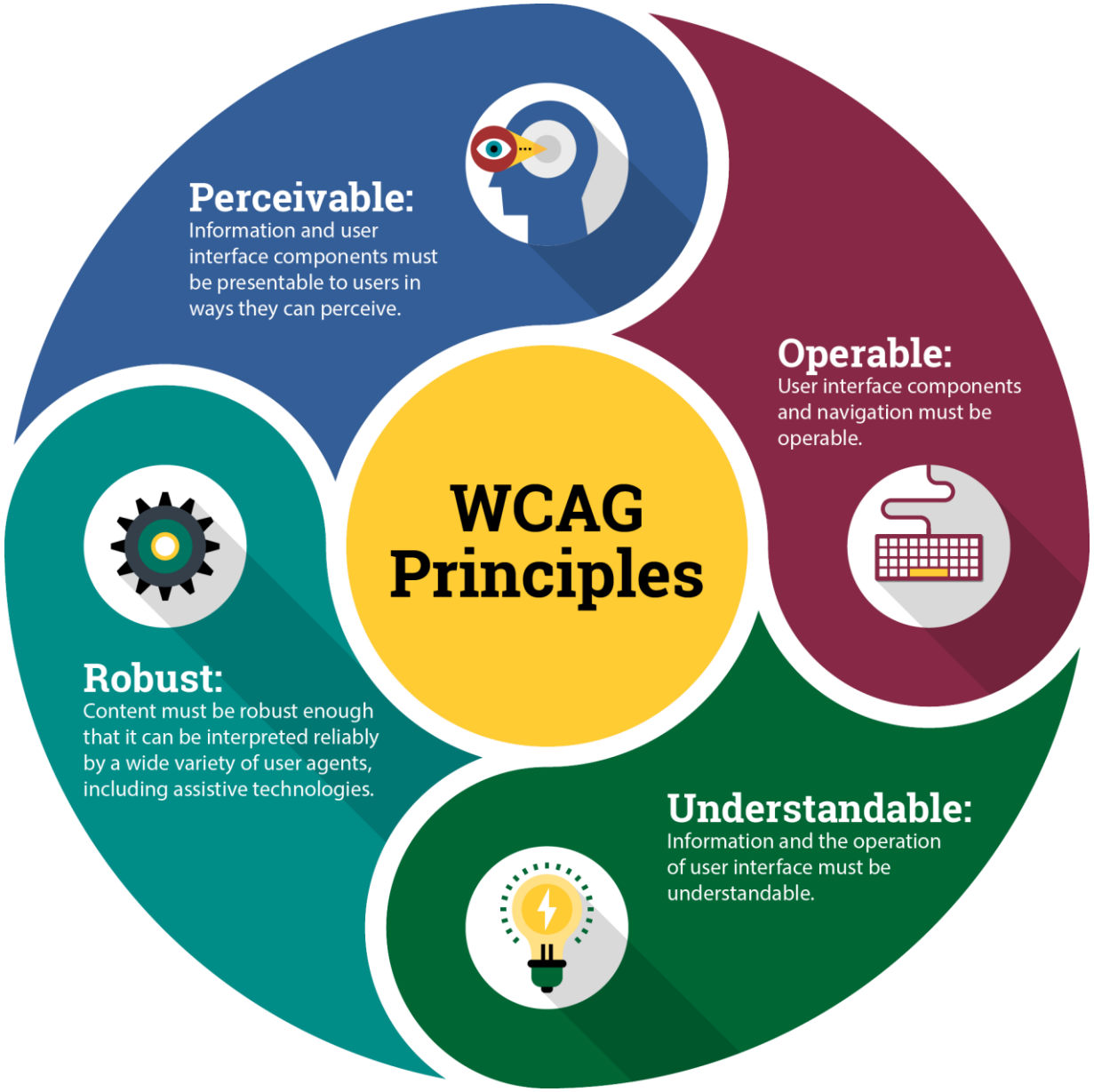



**Level AA includes all Level A requirements + additional criteria**

# The POUR Principles

**Everything in WCAG flows from these four principles:**

- **P – PERCEIVABLE:** *Users must be able to perceive information.* Can they see, hear, or feel your content?
- **O – OPERABLE:** *Users must be able to operate the interface.* Can they navigate and interact with everything?
- **U – UNDERSTANDABLE:** *Users must be able to understand content and interface* Is it clear how things work? Is language readable?
- **R – ROBUST:** *Content must work across technologies.* Does it work with assistive tech, different browsers, future devices?




**Remember POUR—every accessibility requirement fits under one of these!**

# Perceivable




## Perceivable: Users Must Perceive Information

**KEY PRINCIPLE:** Information can't be invisible to all of a user's senses

### REQUIREMENTS:

**Text Alternatives**

- Alt text for images
- Transcripts for audio
- Captions for video
- Descriptions for complex graphics

**Time-Based Media**

- Captions for audio content
- Audio descriptions for visual content
- Sign language (Level AAA)

**Adaptable Content**

- Semantic structure (so screen readers understand)
- Don't rely on visual formatting alone
- Meaningful sequence

**Distinguishable**

- Color contrast: 4.5:1 for normal text, 3:1 for large text
- Don't use color alone to convey information
- Text resizable to 200% without loss of functionality
- Audio control (can pause/stop)

**If users can't perceive it, they can't use it!**

# Operable



**Operable**
User interface components and navigation must be operable

## Operable: Users Must Operate the Interface

**KEY PRINCIPLE:** Interface components and navigation must be operable by everyone

### REQUIREMENTS:

**Keyboard Accessible**
- ALL functionality available via keyboard
- No keyboard trap (can always escape)
- Visible focus indicators

**Enough Time**
- No time limits (or users can extend them)
- Can pause/stop/hide moving content
- Warning before session timeout

**Seizures & Physical Reactions**
- Nothing flashes more than 3 times per second
- No parallax that causes motion sickness

**Navigable**
- Skip links to bypass repetitive content
- Descriptive page titles
- Focus order makes sense
- Link purpose is clear from link text
- Multiple ways to find content

**Input Modalities**
- Works with pointer, keyboard, voice, touch
- Large tap targets (44x44px minimum)
- Motion/gesture alternatives

**Everyone interacts differently—design for all input methods!**

# Understandable



**Understandable**
Information and the operation of user interface must be understandable

## Understandable: Users Must Understand Content

**KEY PRINCIPLE:** Content and interface must be understandable to users

### REQUIREMENTS:

**Readable**

- Language identified (<html lang="en">)
- Clear, simple language (avoid jargon)
- Explain unusual words/abbreviations
- Reading level appropriate for audience
- Pronunciation provided when needed

**Predictable**

- Consistent navigation (same location on every page)
- Consistent identification (icons mean same thing)
- No changes on focus (don't auto-submit forms)
- No surprise page changes
- Predictable behavior patterns

**Input Assistance**

- Clear labels for all form fields
- Error messages are specific and helpful
- Suggestions for fixing errors
- Error prevention for important actions (confirm before delete)
- Instructions provided when needed

**If users don't understand, they can't use it successfully!**

# Robust



**Robust**

Content must be robust enough that it can be interpreted by a wide variety of user agents, including assistive technologies.

## Robust: Content Must Work Everywhere

**KEY PRINCIPLE:** Content must be robust enough to work with current AND future technologies

### REQUIREMENTS:

**Compatible with Assistive Technologies**

- Valid HTML (passes W3C validator)
- Proper use of semantic elements
- Complete start/end tags
- Unique IDs
- ARIA used correctly when needed

**Parse-able**

- HTML that browsers/screen readers can parse
- Well-formed markup
- No duplicate attributes
- Elements properly nested

**Name, Role, Value**

- All UI components have accessible name
- Role is communicated (button, link, checkbox, etc.)
- State is communicated (checked, expanded, selected)

**Status Messages**

- Dynamic updates announced to screen readers
- Use ARIA live regions for updates

**Write clean, semantic HTML = Works with technologies that don't even exist yet!**

# Key Success Criteria

## Must-Have Accessibility Features

**TEXT ALTERNATIVES (1.1.1)**
- Alt text for all informative images
- Empty alt (`alt=""`) for decorative images
- Labels for form inputs

**COLOR CONTRAST (1.4.3)**
- 4.5:1 minimum for normal text
- 3:1 minimum for large text (18pt+ or 14pt+ bold)
- Test EVERY text/background combination!

**KEYBOARD ACCESSIBLE (2.1.1)**
- All functionality available via keyboard
- Logical tab order
- No keyboard traps

**FOCUS VISIBLE (2.4.7)**
- Clear focus indicators on ALL interactive elements
- NEVER remove outline without replacement

**LABELS & NAMES (3.3.2, 4.1.2)**
- Proper `<label>` for every form input
- Connected with `for` and `id` attributes
- Descriptive link text (not "click here")

**ERROR IDENTIFICATION (3.3.1)**
- Errors clearly identified
- Specific, helpful error messages

**SEMANTIC HTML (1.3.1, 4.1.1)**
- Proper heading hierarchy (one H1, logical flow)
- Semantic elements (`<nav>`, `<main>`, `<article>`)
- Valid HTML

**NO KEYBOARD TRAP (2.1.2)**
- Users can always move away from elements

**Master these 8 and you're 80% compliant!**

# Alt Text Example

**THE REQUIREMENT:**

Success Criterion 1.1.1

**RULES:**

- Describe what's IN the image
- Keep it concise (120 chars or less is ideal)
- Don't start with "image of..." (redundant)
- For decorative images, use empty alt: `alt=""`

```
❌ BAD – No Alt Text:
<img src="dog.jpg">
<!-- Screen reader: "Image, dog dot jpg" -->
<!-- Not helpful! Users have no idea what the image shows. -->

❌ BAD – Redundant Alt:
<img src="dog.jpg" alt="image">
<!-- Screen reader: "Image, image" -->
<!-- Still not helpful! "Image" is redundant (screen readers already say "image"). -->

❌ BAD – Filename Alt:
<img src="dog.jpg" alt="dog.jpg">
<!-- Screen reader: "Image, dog dot jpg" -->
<!-- Filename isn't a description! -->

✅ GOOD – Descriptive Alt:
<img src="dog.jpg"
    alt="Golden retriever playing fetch in a park">
<!-- Screen reader: "Image, Golden retriever playing fetch in a park" -->
<!-- Perfect! Users understand what the image shows. -->
```

**Think: If you were describing this image to someone over the phone, what would you say?**

# Color Contrast

## Color Contrast: Make Text Readable

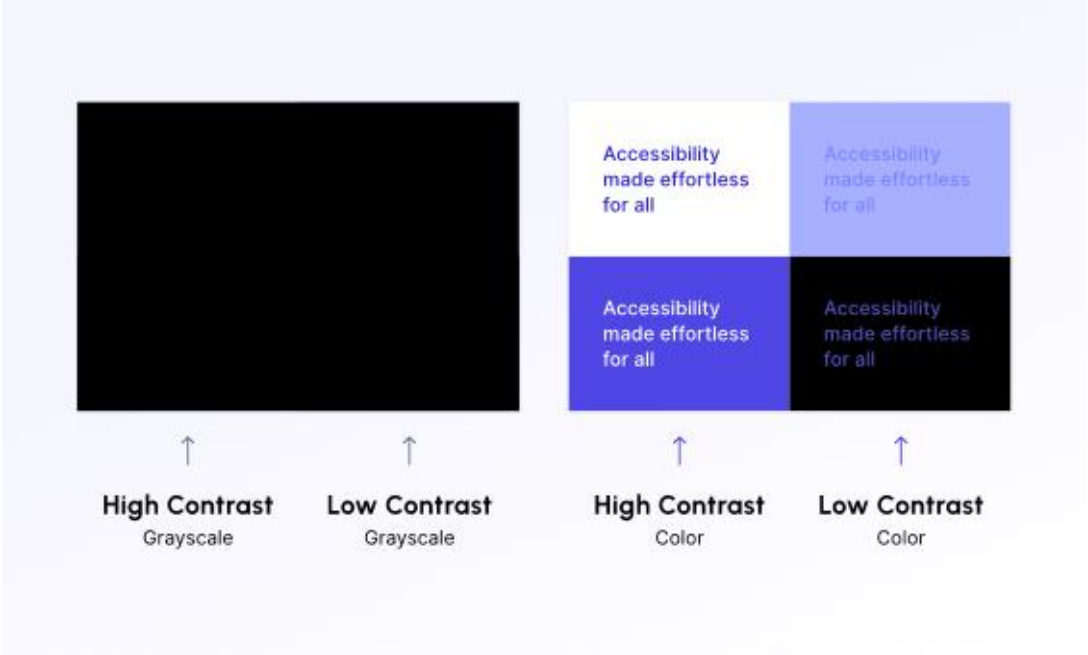
**THE REQUIREMENT:** Success Criterion 1.4.3 (Level AA)

**MINIMUM RATIOS:**
**Normal Text** (under 18pt regular or 14pt bold)

- **4.5:1** minimum ratio
- Example: `#000000` on `#FFFFFF` = 21:1 ✓ (excellent!)
- Example: `#767676` on `#FFFFFF` = 4.54:1 ✓ (just passes)
- Example: `#999999` on `#FFFFFF` = 2.85:1 ✗ (FAILS)

**Large Text** (18pt+ regular or 14pt+ bold)

- **3:1** minimum ratio
- Can be slightly lower contrast since larger text is easier to read




**WHY IT MATTERS:**

- 253 million people have vision impairment
- Aging = vision decline (we ALL get there)
- Bright sunlight makes low contrast unreadable
- Affects reading speed for EVERYONE

**TESTING TOOL: WebAIM Contrast Checker:**
webaim.org/resources/contrastchecker

- Enter foreground and background colors
- Instant pass/fail + suggestions

**Test EVERY text/background combo in your design!**

# Common Contrast Failures

## Common Contrast Mistakes to AVOID

```
/* FAILURE #1: Light Gray on White */
/* FAILS - 2.85:1 */
color: #999999;
background: #FFFFFF;
/* Fix: Use #767676 or darker (4.54:1 ✓) */
```

**FAILURE #1: Light Gray on White**

```
/* FAILURE #2: White Text on Yellow */
/* FAILS - 1.28:1 */
color: #FFFFFF;
background: #FFD700;
/* Fix: Use dark text (#000 or #333) on yellow backgrounds */
```

**FAILURE #2: White Text on Yellow**

**FAILURE #3: "Modern" Low Contrast**

*"But it looks sleek and minimal!"*

- Reality: If users can't read it, it's not good design
- High contrast ≠ harsh. It = accessible and professional

# Common Contrast Failures

```
/* FAILURE #4: Color-on-Color */
/* Blue on Purple - May look different but can fail! */
color: #3498DB; /* Blue */
background: #9B59B6; /* Purple */
/* Ratio: 2.2:1 - FAILS */
/* Fix: Use one color + one neutral (blue on white, white on purple) */
```

**FAILURE #4: Color-on-Color**

```
/* FAILURE #5: Insufficient Link Contrast */
/* Light blue links fail on white */
a { color: #ADD8E6; } /* 1.9:1 - FAILS */
/* Fix: #0066CC = 7.2:1 ✓ */
```

**FAILURE #5: Insufficient Link Contrast**

**Use WebAIM Contrast Checker before you commit to colors!**

# Why Semantic HTML?

## Semantic HTML = Free Accessibility

**What is semantic HTML?**
HTML elements that have **meaning** beyond just presentation.

- **<header>** means 'this is the header area'
- **<nav>** means 'this is navigation'
- **<main>** means 'this is the main content'
- **<article>** means 'this is an article'

**Screen reader hears:**
- "Banner landmark",
- "Navigation landmark",
- "Main landmark",
- "Article",
- "Heading level 1: Article Title"

```
<header>
    <nav>
        <a href="/">Home</a>
    </nav>
</header>
<main>
    <article>
        <h1>Article Title</h1>
        <p>Content here...</p>
    </article>
</main>
<footer>
    <p>&copy; 2026</p>
</footer>
```

**Semantic HTML = Assistive tech understands your page structure!**

# Semantic Structure

## GOOD: Semantic HTML

**Benefits:**

- Clear semantic structure
- Proper interactive elements (links)
- Screen readers understand everything
- Landmarks for easy navigation
- SEO-friendly
- Maintainable

```
<header>
    <h1>Site Name</h1>
    <nav>
        <ul>
            <li><a href="/">Home</a></li>
            <li><a href="/about">About</a></li>
        </ul>
    </nav>
</header>
<main>
    <article>
        <h2>Article Title</h2>
        <p>Content...</p>
    </article>
</main>
<footer>
    <p>Contact info</p>
</footer>
```

**Use the RIGHT element for the job!**

# Semantic Structure

**KEY SEMANTIC ELEMENTS YOU SHOULD USE:**

**Page structure:**

- `<header>` - Page header
- `<nav>` - Navigation menu
- `<main>` - Main content (one per page!)
- `<aside>` - Sidebar or related content
- `<footer>` - Page footer

**Content structure:**

- `<article>` - Self-contained content (blog post, news article)
- `<section>` - Thematic grouping of content
- `<h1>` - `<h6>` - Headings (we'll cover hierarchy next)

**Text semantics:**

- `<p>` - Paragraphs
- `<ul>`/`<ol>`/`<li>` - Lists
- `<strong>` - Important text (not just bold!)
- `<em>` - Emphasized text (not just italic!)

**Interactive:**

- `<a href="...">` - Links (go somewhere)
- `<button>` - Buttons (do something)
- `<form>` - Forms
- `<label>` - Form labels

**WHEN TO USE DIV: Divs aren't evil! They have a purpose: styling containers with no semantic meaning.**

# Heading Hierarchy

**BAD: Broken Hierarchy**

```
<h1>Main Page Title</h1>
<h3>Wait, where's H2?</h3> <!-- Skipped H2! -->
<h2>Now H2 appears</h2> <!-- Out of order! -->
<h1>Another H1</h1> <!-- Multiple H1s! -->
<h4>Random H4</h4> <!-- Skipped H3! -->
```

**GOOD: Logical Hierarchy**

```
<h1>Main Page Title</h1> <!-- Page title, only one -->
  <h2>Major Section</h2>
    <h3>Subsection</h3>
    <h3>Another Subsection</h3>
  <h2>Another Major Section</h2>
    <h3>Subsection Here</h3>
      <h4>Detail Level</h4>
```

**THE RULE:**

- **One `<h1>` per page** (the main page title)
- **Never skip heading levels** (H1 → H2 → H3, not H1 → H3)
- **Headings create document outline** (screen readers navigate by headings)

**Think of headings like a table of contents!**

# Descriptive Links

**THE PROBLEM:** Screen reader users navigate by links—they hear a LIST of all links on the page out of context.

**TERRIBLE: Generic Link Text**

```
<a href="report.pdf">Click here</a>
<a href="article.html">Read more</a>
<a href="guide.pdf">Download</a>
<a href="/about">Learn more</a>
```

**User thinks:** "Click here for WHAT? Read more about WHAT? Download WHAT?"

**BONUS TIPS:**

- Include file type for PDFs/docs: (PDF) (Word doc)
- Include file size for downloads: (2MB)
- Indicate external links: (opens in new window)
- **Link text should make sense OUT of context!**

**GOOD: Descriptive Link Text**

```
<a href="report.pdf">Download Annual Report (PDF, 2MB)</a>
<a href="article.html">Read our accessibility guide</a>
<a href="guide.pdf">Download beginner's HTML guide (PDF)</a>
<a href="/about">Learn more about our team</a>
```

**Links are a major navigation tool for screen reader users. Let's make sure yours are accessible.**

# Buttons vs Links

**WHY IT MATTERS:**

- Screen readers announce them differently
- Keyboard behavior is different (Enter vs Space)
- Browser features differ (right-click to open in new tab works on links, not buttons)
- User expectations differ

**BUTTON = Does Something**

- Submits a form
- Opens/closes something
- Triggers an action
- Changes state

```
<button type="submit">Submit Form</button>
<button onclick="openMenu()">Open Menu</button>
<button onclick="toggleDarkMode()">Toggle Dark Mode</button>
<button onclick="addToCart()">Add to Cart</button>
```

**LINK = Goes Somewhere**

- Navigates to a page
- Jumps to an anchor
- Downloads a file
- Changes URL

```
<a href="/about">About Us</a>
<a href="report.pdf">Download Report</a>
<a href="#contact">Jump to Contact Section</a>
<a href="https://example.com">Visit Example</a>
```

**Use the RIGHT element for the RIGHT job!**

# Form Labels

**THE REQUIREMENT:** Every form input must have a properly connected label (Success Criterion 3.3.2)

**WHY LABELS MATTER:**

- **For screen reader users:** When they focus an input, the screen reader announces the label. No label = they don't know what to type.
- **For cognitive disabilities:** Clear labels help users understand what's expected.
- **For motor disabilities:** Clicking the label focuses the input, making it a larger click target.
- **For EVERYONE:** Labels make forms understandable!

```
BAD: Placeholder as Label
<input type="email" placeholder="Email Address">
```

**Problems:**

- Placeholder disappears when user types
- Not reliably accessible to screen readers
- Low contrast (usually too light)
- Can't see label while filling out field

```
BAD: No Connection Between Label and Input
<label>Email Address</label>
<input type="email">
```

```
GOOD: Proper Label Connection
<label for="email">Email Address</label>
<input type="email" id="email" name="email">
```

**RULE: Label must be connected with for/id attributes!**

# Image Alt Text Rules

```
RULE #1: DECORATIVE IMAGES → Empty Alt
<!-- Background patterns, spacers, purely aesthetic -->
<!-- Empty alt tells screen readers to skip it -->
<img src="decorative-pattern.png" alt="">
```

```
RULE #2: INFORMATIVE IMAGES → Descriptive Alt
<!-- Describe what's important about the image -->
<!-- Photo with useful information -->
<img src="chart.png"
     alt="Bar chart showing 40% sales increase in Q4 2025">

<!-- Product photo -->
<img src="laptop.jpg"
     alt="Silver MacBook Pro with 16-inch display">
```

**Ask: If I described this image over the phone, what would I say?**

# Image Alt Text Rules

```
RULE #3: FUNCTIONAL IMAGES → Describe Action
<!-- Describe what happens when user interacts -->
<!-- Logo that's also a link -->
<a href="/">
  <img src="logo.png" alt="Return to homepage">
</a>

<!-- Icon button -->
<button>
  <img src="search-icon.svg" alt="Search">
</button>
```

**ALT TEXT BEST PRACTICES:**

- Concise (120 chars or less when possible)
- Context-appropriate (same image, different contexts = different alt)
- Don't start with "image of" or "picture of"
- Include text that appears in image
- Don't be redundant with surrounding text
- Don't be overly descriptive for simple images

```
RULE #4: COMPLEX IMAGES → Brief Alt + Long Description
<!-- Detailed infographic -->
<img src="infographic.png"
     alt="2025 web accessibility statistics infographic"
     aria-describedby="infographic-desc">

<div id="infographic-desc">
  <p>Detailed description: The infographic shows...</p>
</div>
```

**Ask: If I described this image over the phone, what would I say?**

# What is ARIA?

## ARIA: When HTML Needs Extra Help

**WHAT IS ARIA? Accessible Rich Internet Applications**
A set of HTML attributes that provide additional accessibility information when semantic HTML isn't enough

```
<!-- DO THIS -->
<nav>...</nav>
<button>...</button>
<header>...</header>

<!-- NOT THIS -->
<div role="navigation">...</div>
<div role="button">...</div>
<div role="banner">...</div>
```

**WHEN TO USE ARIA:**

**1. When no semantic HTML exists**

- Modal dialogs: `role="dialog"`
- Alert messages: `role="alert"`
- Tab interfaces: `role="tablist"`

**2. To add extra context**

- Icon buttons: `aria-label="Close"`
- State: `aria-expanded="true"`
- Relationships: `aria-describedby="help-text"`

**3. For dynamic content**

- Live regions: `aria-live="polite"`
- Loading states: `aria-busy="true"`

**ARIA never fixes bad HTML—it only adds information to good HTML**

# Categories of ARIA

**1. ROLES** - Define what an element is
- `role="button"` (though use `<button>` instead!)
- `role="alert"` (for important messages)
- `role="dialog"` (for modal windows)

**2. STATES** - Define current state
- `aria-expanded="true"` (is this expanded?)
- `aria-checked="true"` (is this checked?)
- `aria-pressed="true"` (is this toggle pressed?)

**3. PROPERTIES** - Define characteristics
- `aria-label="Close"` (accessible name)
- `aria-describedby="help"` (connected description)
- `aria-labelledby="title"` (connected label)




**ARIA never fixes bad HTML—it only adds information to good HTML**

# ARIA Roles

**ARIA Roles: When Semantic HTML Doesn't Exist**

**ROLES TO KNOW:**

- **alert** - Important, time-sensitive message
- **dialog** - Modal dialog window
- **status** - Status indicator that updates
- **tablist/tab/tabpanel** - Tab interface
- **button** - Button (but use `<button>`!)
- **link** - Link (but use `<a href>`!)

```
Document Structure:
<!-- Alert messages -->
<div role="alert">
  Form submitted successfully!
</div>

<!-- Status updates -->
<div role="status">
  Saving...
</div>
```

```
Interactive Widgets:
<!-- Dialog/Modal -->
<div role="dialog" aria-labelledby="dialog-title">
  <h2 id="dialog-title">Confirm Delete</h2>
  <p>Are you sure?</p>
  <button>Cancel</button>
  <button>Delete</button>
</div>

<!-- Tab interface -->
<div role="tablist">
  <button role="tab" aria-selected="true">Tab 1</button>
  <button role="tab">Tab 2</button>
</div>
<div role="tabpanel">Content for Tab 1</div>
```

**ARIA roles are a last resort, not a first choice**

# When You Actually Need Roles?

```
ALERT ROLE:
<!-- For important messages that need immediate attention: -->
<div role="alert">
  Your session will expire in 2 minutes
</div>
```

**Why role="alert"?**

- Screen readers announce it immediately (interrupts current reading)
- Like an assertive announcement

**Use for:** Error messages, urgent warnings, success confirmations
**Don't overuse!** Only for truly important messages.

```
STATUS ROLE:
<!-- For status updates that aren't urgent: -->
<div role="status">
  Saving document...
</div>
```

**Why role="status"?**

- Screen readers announce it when user is idle (doesn't interrupt)
- Polite announcement

**Use for:** Loading indicators, save status, progress updates

**ARIA roles are a last resort, not a first choice**

# ARIA Labels

**ARIA Labels: Adding Accessible Names**

**WHEN TO USE aria-label:**

```
Icon buttons:
<button aria-label="Search">
  <svg><!-- magnifying glass --></svg>
</button>

<button aria-label="Menu">
  ≡
</button>

<button aria-label="Save">
  💾
</button>
```

```
Social media icons:
<a href="twitter.com/user" aria-label="Follow us on Twitter">
  <svg><!-- Twitter logo --></svg>
</a>
```

```
Providing additional context:
<button aria-label="Delete item: Blue Sweater">
  Delete
</button>
```

**RULE: Only use aria-label when there's no visible text!**

# ARIA-LABELLEDBY

**ARIA-LABELLEDBY: Reference Existing Text**

```
<div role="dialog" aria-labelledby="dialog-title">
  <h2 id="dialog-title">Confirm Action</h2>
  <p>Are you sure?</p>
</div>
Dialog is labeled by the heading text
```

```
Instead of duplicating text in aria-label, you can reference existing text:
<div role="dialog" aria-labelledby="dialog-title">
  <h2 id="dialog-title">Confirm Delete</h2>
  <p>Are you sure you want to delete this item?</p>
</div>
```

**aria-labelledby="dialog-title"** tells screen readers: 'This dialog's label is the element with id="dialog-title"'

**Benefits:**

- No duplicate text
- If heading changes, aria-label updates automatically
- Cleaner code

**RULE: Only use aria-label when there's no visible text!**

# ARIA States

**ARIA States: Communicate What's Happening**

**THE PROBLEM:** Interactive elements have states that change—screen readers need to know!

**COMMON ARIA STATES:**

```
aria-expanded - Is this expanded or collapsed?
<!-- Collapsed -->
<button aria-expanded="false" aria-controls="menu">
  Menu
</button>
<div id="menu" hidden>...</div>

<!-- Expanded -->
<button aria-expanded="true" aria-controls="menu">
  Menu
</button>
<div id="menu">...</div>
```

```
aria-pressed - Is this toggle button pressed?
<!-- Not pressed -->
<button aria-pressed="false">
  Toggle Dark Mode
</button>

<!-- Pressed -->
<button aria-pressed="true">
  Toggle Dark Mode
</button>
```

**States must change dynamically to reflect current status**

# ARIA States

**COMMON ARIA STATES:**

```
aria-checked - Is this checked? (for custom checkboxes)
<!-- Use real checkbox when possible! -->
<input type="checkbox" checked> <!-- Best -->

<!-- Only if you must use custom -->
<div role="checkbox" aria-checked="true">
  Subscribe to newsletter
</div>
```

```
aria-selected - Is this selected? (for tabs, options)
<button role="tab" aria-selected="true">Tab 1</button>
<button role="tab" aria-selected="false">Tab 2</button>
```

**States must change dynamically to reflect current status**

# ARIA States

**COMMON ARIA STATES:**

```
aria-invalid - Is this form field invalid?
<input
  type="email"
  aria-invalid="true"
  aria-describedby="email-error">
<span id="email-error">Invalid email format</span>
```

```
aria-current - What's the current page/step?
<nav>
  <a href="/" aria-current="page">Home</a>
  <a href="/about">About</a>
  <a href="/contact">Contact</a>
</nav>
```

**States must change dynamically to reflect current status**

# ARIA Live Regions

**ARIA Live Regions: Announce Dynamic Changes**

**THE PROBLEM:** When content updates without page refresh, screen readers don't notice

**TWO POLITENESS LEVELS:**

```
aria-live="polite" - Wait until user is idle
<div aria-live="polite">
  Saving your changes...
</div>
```

- **Use for:** Status updates, cart counters, non-urgent messages
- **Behavior:** Announces when user pauses

```
aria-live="assertive" - Interrupt immediately
<div aria-live="assertive">
  Error: Form submission failed
</div>
```

- **Use for:** Critical errors, urgent warnings
- **Behavior:** Interrupts current reading

**RULE: Announce changes users need to know about!**

# ARIA Live Regions

**ALTERNATIVES**

```
ALTERNATIVE: ROLE="ALERT"
<div role="alert">
  Form submitted successfully!
</div>
```

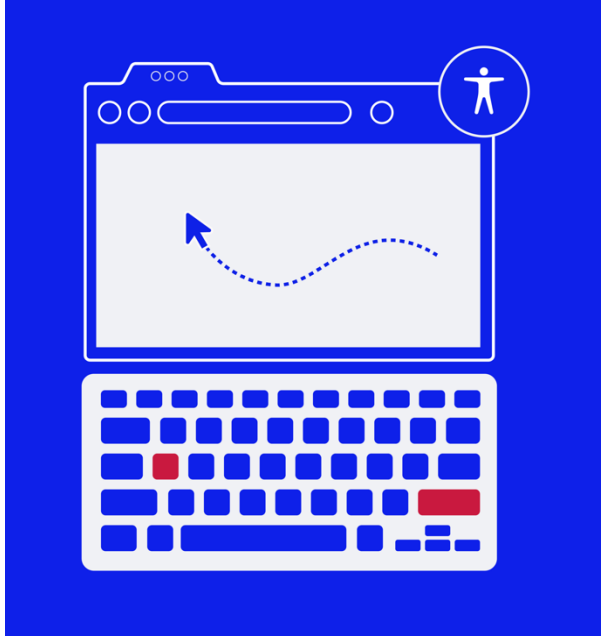
**Same as aria-live="assertive"** (interrupts immediately)

```
ALTERNATIVE: ROLE="STATUS"
<div role="status">
  Loading content...
</div>
```

**Same as aria-live="polite"** (waits until idle)

**RULE: Announce changes users need to know about!**

# Why Keyboard Access?



**NOT EVERYONE USES A MOUSE:**

**WHO NEEDS KEYBOARD ACCESS?**

**Blind Users**

- Use screen readers + keyboard
- Never see mouse cursor
- Navigate entirely by Tab/Arrow keys

**Motor Disability Users**

- Can't control mouse precisely
- May use keyboard, switch device, or voice control
- Need large, keyboard-accessible targets

**Power Users**

- Prefer keyboard (it's faster!)
- Use shortcuts extensively
- Navigate without touching mouse

**Switch Device Users**

- Use single button (head, foot, etc.)
- Simulates keyboard Tab/Enter
- Need perfect tab order

**Voice Control Users**

- Say "Click Submit"
- Relies on keyboard-accessible elements
- Buttons/links must be properly labeled

**THE REQUIREMENT: ALL functionality must work with keyboard alone**

# ALL functionality must work with keyboard alone

**THIS MEANS:**

- Can reach every link, button, form field
- Can activate every interactive element
- Can open/close menus and modals
- Can submit forms
- Can navigate through tabs
- Can escape from components (no traps!)

**KEYBOARD BASICS:**

- **Tab** - Move forward
- **Shift + Tab** - Move backward
- **Enter** - Activate links and buttons
- **Space** - Activate buttons, toggle checkboxes
- **Escape** - Close modals/menus
- **Arrow keys** - Navigate within components

**TEST YOUR ENTIRE SITE WITH JUST KEYBOARD!**

# Keyboard Accessible Elements

**NATURALLY KEYBOARD ACCESSIBLE:**

```
These elements work with keyboard by default—USE THEM!
<button>Click Me</button>
<!-- Tab to focus, Enter/Space to activate -->

<a href="/page">Link</a>
<!-- Tab to focus, Enter to activate -->

<input type="text">
<input type="email">
<input type="checkbox">
<input type="radio">
<select>...</select>
<textarea>...</textarea>
<!-- All form elements: Tab to focus, type/interact normally -->
```

**ALWAYS use <button> and <a href> for interactive elements!**

# Focus Indicators

**Focus Indicators: Show Users Where They Are**

**Option 1: Use Browser Default**

```css
/* Let default outline show - it works! */
button:focus {
  /* Don't remove outline */
}
```

**Option 2: Custom Focus Style**

```css
button:focus {
  outline: 3px solid #3498db;
  outline-offset: 2px;
}

/* Or use box-shadow */
button:focus {
  outline: none; /* OK if replacing with something visible */
  box-shadow: 0 0 0 3px rgba(52, 152, 219, 0.5);
}
```

**Option 3: :focus-visible (Modern)**

```css
/* Style focus only for keyboard users */
button:focus-visible {
  outline: 3px solid #3498db;
  outline-offset: 2px;
}
```

**REQUIREMENTS:**

- Visible on ALL interactive elements
- High contrast (3:1 minimum with background)
- Clear and obvious
- Consistent across site

**WCAG 2.4.7: Focus must be visible! No exceptions!**

# Tab Order

**Tab Order: Logical Flow Matters**

**THE RULE:** Tab order should follow logical reading order (typically top-to-bottom, left-to-right)

**GOOD: Natural DOM Order**

```
<header>
  <nav>
    <a href="/">Home</a>      <!-- Tab 1 -->
    <a href="/about">About</a> <!-- Tab 2 -->
  </nav>
</header>
<main>
  <button>Search</button>      <!-- Tab 3 -->
  <input type="text">          <!-- Tab 4 -->
</main>
```

**TABINDEX VALUES:**

```
<!-- tabindex="0" - Include in natural tab order -->
<div role="button" tabindex="0">Custom Button</div>
<!-- tabindex="-1" - Remove from tab order (focusable by script) -->
<div id="error" tabindex="-1">Error message</div>
```

**tabindex="1+" (positive numbers)** - DON'T USE! *Breaks natural order, creates confusion*

- Only use tabindex="0" to make non-interactive elements focusable
- Only use tabindex="-1" to remove from tab order NEVER use positive tabindex values!

**RULE: Let natural DOM order handle tab sequence!**

# Skip Links

**Skip Links: Jump Past Repetitive Content**

**THE PROBLEM:** Keyboard users must tab through entire navigation on EVERY page

**THE SOLUTION: Skip Link**

```html
<body>
  <a href="#main" class="skip-link">
    Skip to main content
  </a>

  <header>
    <nav>
      <!-- 20 navigation links -->
    </nav>
  </header>

  <main id="main" tabindex="-1">
    <!-- Main content here -->
  </main>
</body>
```

**CSS to hide until focused:**

```css
.skip-link {
  position: absolute;
  top: -40px;
  left: 0;
  background: #000;
  color: white;
  padding: 8px;
  text-decoration: none;
  z-index: 100;
}

.skip-link:focus {
  top: 0;
}
```

**HOW IT WORKS:**

1. First Tab press → Skip link appears
2. Press Enter → Jumps to main content
3. Next Tab → First interactive element in main content

**BEST PRACTICES:**

- First focusable element on page
- Links to id="main" or main content area
- Visible when focused
- High contrast
- Clear text: "Skip to main content" (not just "Skip")

**Skip links are WCAG Level A requirement for sites with repeated content!**

# Essential Keyboard Keys

## Keyboard Keys Users Rely On

**NAVIGATION:**

- **Tab** - Move forward to next interactive element
- **Shift + Tab** - Move backward to previous element

**ACTIVATION:**

- **Enter** - Activate links, submit forms, press buttons
- **Space** - Activate buttons, toggle checkboxes, scroll page

**ESCAPE:**

- **Escape** - Close modals, dismiss menus, cancel actions

**COMPONENT NAVIGATION:**

- **Arrow keys** - Navigate radio groups, select options, tabs
- **Home/End** - Jump to first/last item in lists
- **Page Up/Down** - Scroll content

**FORMS:**

- **Type normally** in text inputs
- **Space** to check/uncheck boxes
- **Arrow keys** for radio buttons
- **Arrow/Type** in dropdowns

**TEST YOUR SITE: Can you complete every task using only these keys?**

# Keyboard Testing Checklist

## TEST YOUR SITE WITH KEYBOARD ONLY:

**BASIC NAVIGATION:**

- ☐ Can reach ALL interactive elements (links, buttons, inputs)?
- ☐ Focus visible at all times?
- ☐ Tab order logical and predictable?
- ☐ No unexpected focus jumps?

**ACTIVATION:**

- ☐ Links activate with Enter?
- ☐ Buttons activate with Enter AND Space?
- ☐ Form submission works?

**FORMS:**

- ☐ Can tab to every form field?
- ☐ Can type in text inputs?
- ☐ Can select radio/checkbox with Space?
- ☐ Can navigate dropdown with arrows?
- ☐ Can submit form with keyboard?

**INTERACTIVE COMPONENTS:**

- ☐ Menus open/close with keyboard?
- ☐ Modals close with Escape?
- ☐ No keyboard traps (can always escape)?
- ☐ Skip link works?

**RULE: If you can't do it with keyboard, keyboard users can't either!**

# Testing Tools

## RECOMMENDED TOOLS:

**1. WAVE (WebAIM)** - Browser Extension
- Visual feedback (icons overlay your page)
- Shows errors, alerts, structural elements
- Contrast checker built-in
- **FREE and beginner-friendly**
- Install: wave.webaim.org/extension

**2. axe DevTools** - Browser Extension
- Comprehensive automated testing
- Detailed explanations of issues
- Suggests fixes
- Industry standard
- Install: chrome.com/webstore (search "axe DevTools")

**3. Lighthouse** - Built into Chrome DevTools
- Accessibility score (0-100)
- Performance, SEO, best practices too
- No installation needed!
- Run: Chrome DevTools → Lighthouse tab

**WHAT AUTOMATED TOOLS FIND:**
- Missing alt text
- Color contrast failures
- Missing form labels
- Invalid HTML
- Missing ARIA attributes
- Heading hierarchy issues

**USE AUTOMATED TOOLS + MANUAL TESTING**

# WAVE Demo

**INSTALL:** wave.webaim.org/extension

**HOW TO USE:**

1. Visit your webpage
2. Click WAVE extension icon
3. Icons appear on page showing issues

**ICON MEANINGS:**

**ERRORS (Must Fix!)**

- Missing alt text
- Missing form labels
- Empty links/buttons
- Color contrast failures

**FEATURES (Good!)**

- Alt text present
- ARIA labels
- Semantic structure
- Skip links

**ALERTS (Check These)**

- Suspicious alt text
- Redundant links
- Possible heading issues
- Layout tables

**STRUCTURAL ELEMENTS**

- Headings (H1-H6)
- Lists
- Landmarks (nav, main, etc.)

**CONTRAST TAB:**

- Test any text/background combo
- Instant pass/fail
- Suggested fixes

**Don't wait until the end! Run WAVE regularly as you build.**

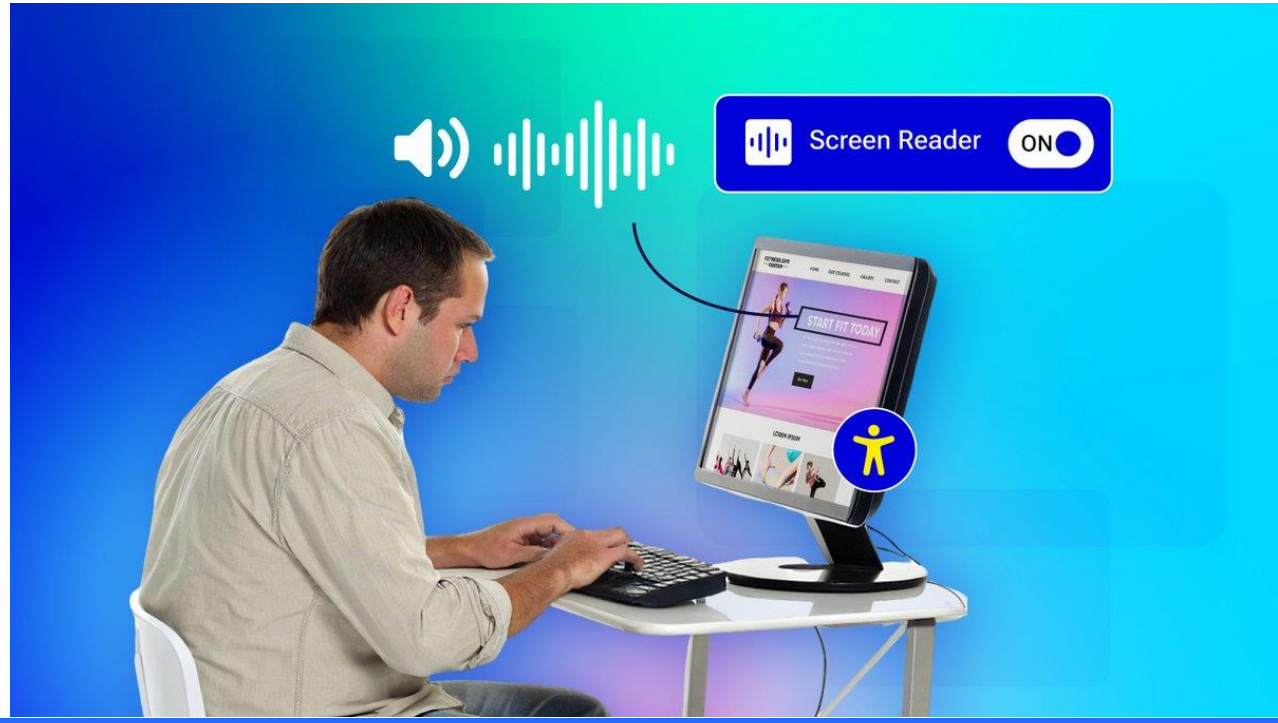
# Screen Reader Demo

**FREE OPTIONS:**

- **NVDA** (Windows) - nvaccess.org
- **JAWS** (Windows) - Paid, but industry standard
- **VoiceOver** (Mac) - Built-in! Cmd+F5 to enable
- **TalkBack** (Android) - Built-in
- **VoiceOver** (iOS) - Built-in

**WHAT THEY DO:**

- Read all text on page aloud
- Announce element types ("link", "button", "heading")
- Allow navigation by landmarks, headings, links
- Read alt text for images
- Announce form labels




**Hearing a screen reader changes everything!**


ADA, WCAG?
Semantic HTML?
POUR principles?
ARIA?
Test with keyboard?
Anything unclear?
HAVE ANY
QUESTIONS?